

ICLR 2026

CFO: LEARNING CONTINUOUS-TIME PDE DYNAMICS VIA FLOW-MATCHED NEURAL OPERATORS

Xianglong Hou

Graduate Group in Applied Mathematics and Computational Science

University of Pennsylvania

Philadelphia, PA 19104

`xlhou@sas.upenn.edu`

Xinquan Huang, Paris Perdikaris

Department of Mechanical Engineering and Applied Mechanics

University of Pennsylvania

Philadelphia, PA 19104

`{huang26,pgp}@seas.upenn.edu`

2026.6.18 张欣宇

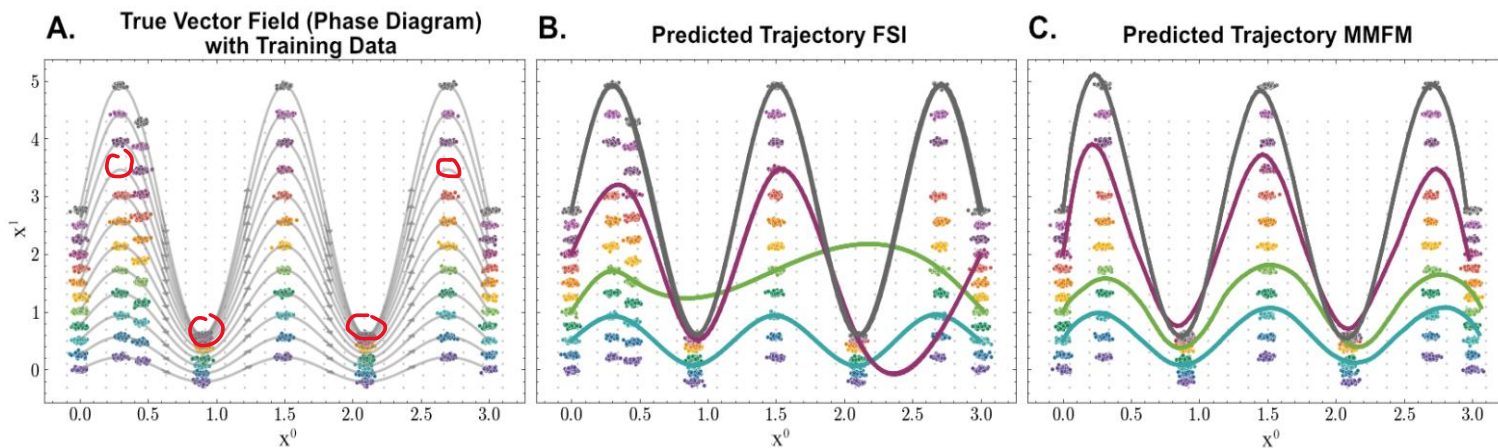
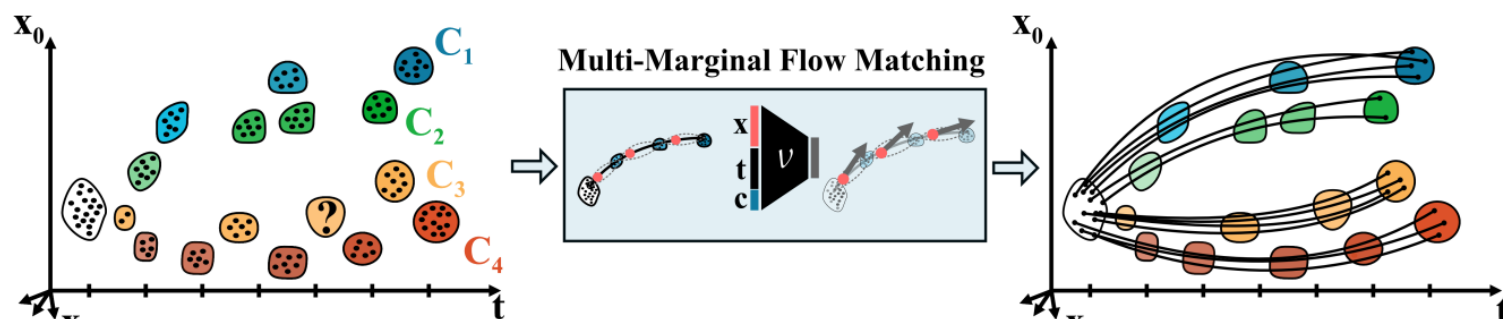
回顾 MMFM (Multi-Marginal Flow Matching) :

任务目标: 给定多个时刻的稀疏数据快照 $\rightarrow$ 恢复整条连续轨迹

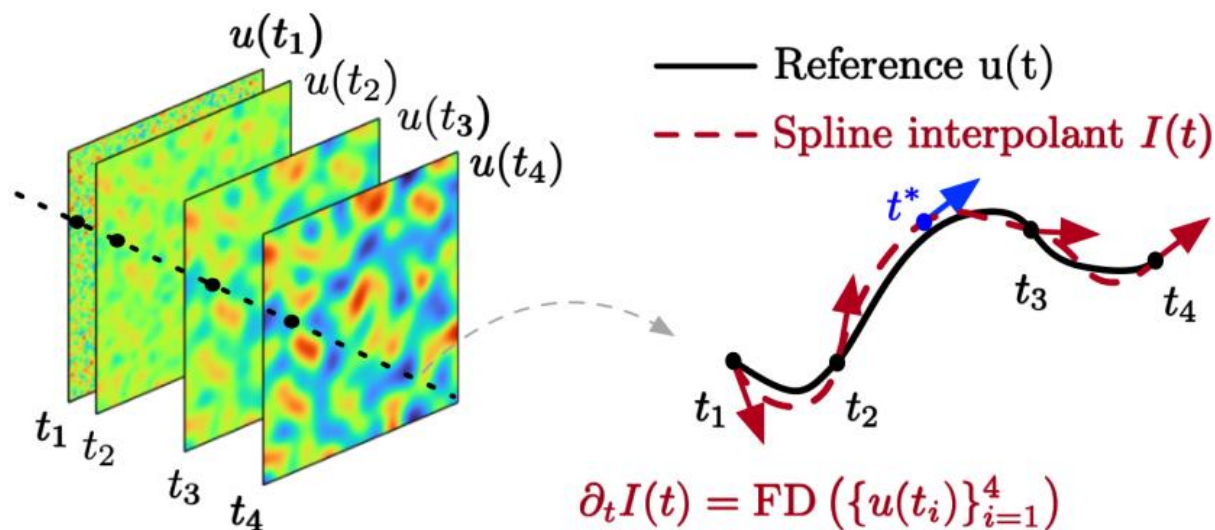
把 FM 的"两端分布映射"推广到"多个边缘分布之间的联合流"

具体方法: 三次样条作为轨迹先验 + FM 学条件向量场

(多个时刻的边缘分布都被流穿过)



CFO: Learning Continuous-Time PDE Dynamics via Flow-Matched Neural Operators (ICLR 2026)



核心方法:

五次样条插值+FM 学条件向量场+ RK4 数值积分工具

Flow Matching 的训练范式:

'噪声→数据'这条人造路径



PDE 的'物理时间轨迹'这条真实路径上来



纯数据驱动方式学连续时间 PDE 求解

思路与实现简单, 可以换个视角:

- FM 训练范式可以通过'路径设计'参数化
- Flow: 有“真实物理时间维”的概率路径框架、应用场景可拓展

背景：现有 PDE 神经求解三大流派

(1) Neural PDE Solvers & Operator Learning

FNO：在 Fourier 域做卷积，学算子 $u_0 \mapsto u_T$

DeepONet：branch-trunk 网络学算子

U-NO, Geo-FNO：FNO 的几何/多分辨率扩展

FNO也可以设计成滚动的AR形式：

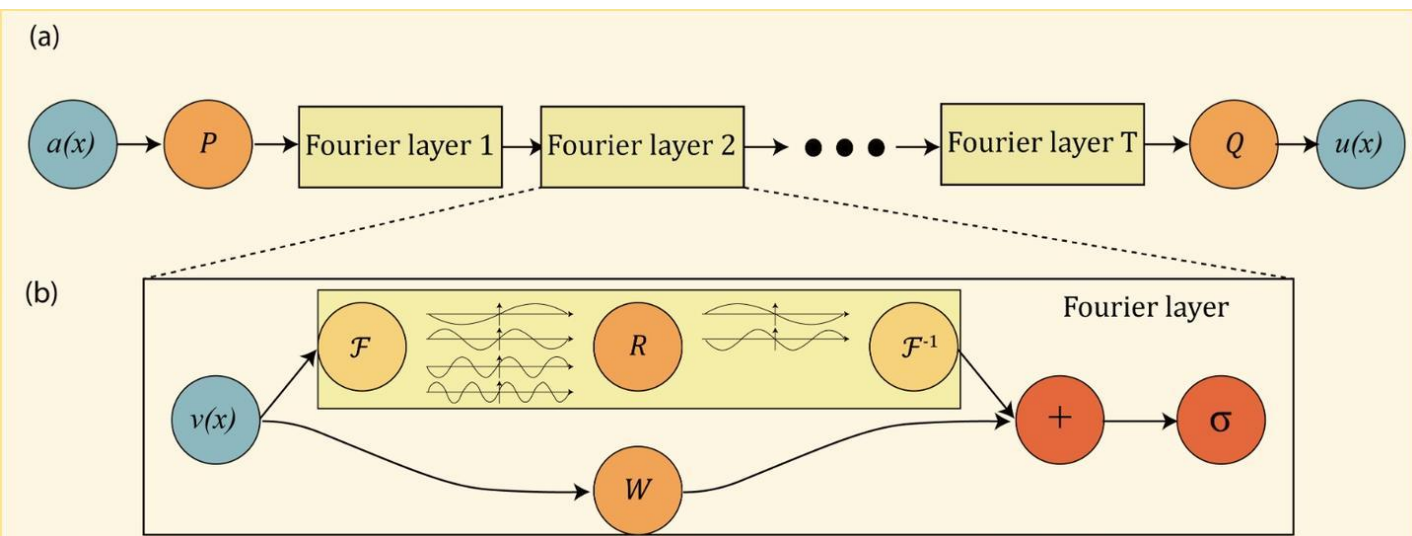
$$u(x, t + \Delta t) = \mathcal{G}_\theta(u(x, t))$$

优点：

- 推理快（单次前向）
- 可学高维 PDE（FNO 用 FFT 加速）

问题：

- 离散时间网格固定（只能学习见过的t）
（训练 $T=100 \rightarrow$ 推理也必须 $T=100$ ）
- 时间分辨率不可变（ Δt 固定）
- 长时程外推困难（AR rollout 误差累积）
- 不显式建模动力学（黑盒映射）



FNO为例：

1. 对输入进行升维，具体来说是从2维升到64维；
2. 经过多个(4个)傅立叶层
3. 降维，得到一个数值，即对应 $u_0 \mapsto u_T$ 的解。

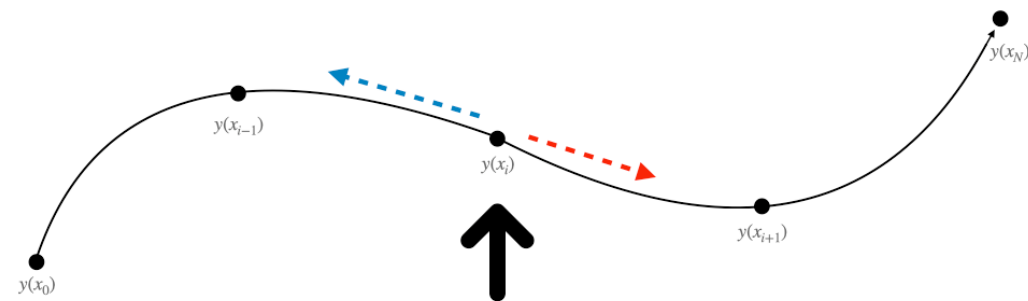
背景：现有 PDE 神经求解三大流派

(2) Neural ODE (CNF)

Neural ODE : $\dot{h} = f_{\theta}(h, t)$

PINODE : 物理嵌入 NODE

Neural CDE : irregular time series



标准 CNF 的训练数据是配对形式：($u'(0) \rightarrow u(T)$)。

神经网络当成 ODE 右端项描述隐藏状态 $h(t)$ 是如何随时间 t 演化 (Flow)

Flow 方法：支持 forward 与 backward

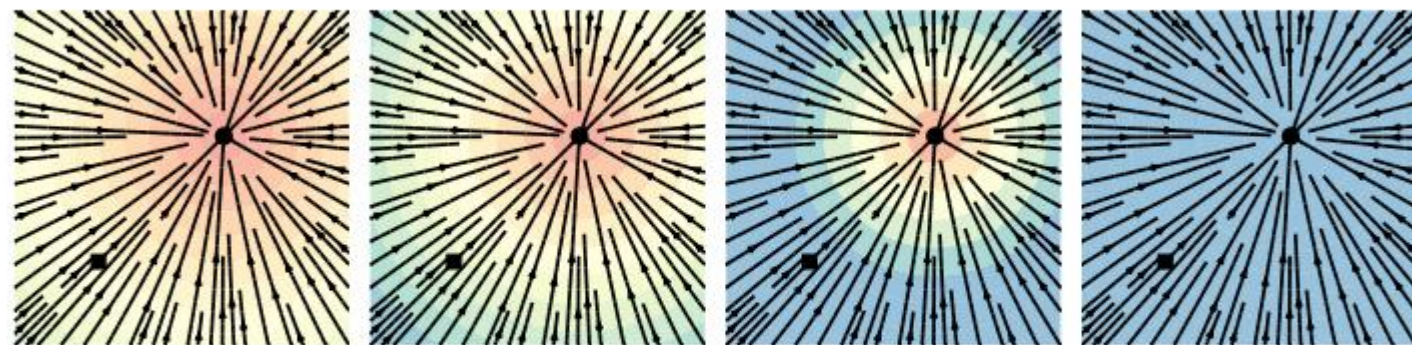
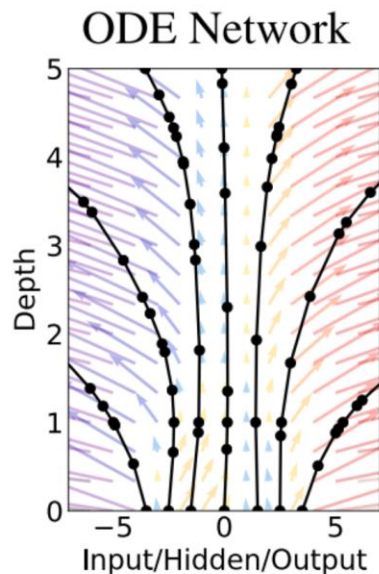
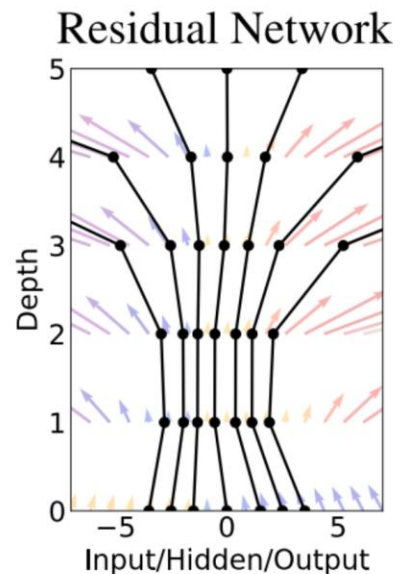
$$\frac{dh(t)}{dt} = f_{\theta}(h, (t)t), h(t_0) = h_0$$

推理 (forward) : 标准 ODE 数值积分

$$h(t_N) = h(t_0) + \int_{t_0}^{t_N} f_{\theta}(h, (\tau)\tau) d\tau \approx \text{ODESolver}(f_{\theta}, h_0, [t_0, t_N])$$

损失函数： $L(\theta) = \|u_{\theta}(T) - u_{true}(T)\|^2$

训练慢： 每 step 都要 forward + adjoint backward 各解一次 ODE



$t = 0.0$

$t = 1/3$

$t = 2/3$

$t = 1.0$

OT path – conditional vector field

都是ODE连续变换实现建模分布映射

维度
模型
推理
轨迹本身
训练

Neural ODE

$v_{\theta}(x, t)$ (向量场)

ODE 数值积分

一条能完成任务的路径 (可能不符合规律)

Backprop through ODE (adjoint)

Flow Matching

$v_{\theta}(x, t)$ (向量场) ← 相同

ODE 数值积分 ← 相同

是定义的最优概率路径

直接 MSE 回归到速度目标

$$\mathcal{L}_{FM}(\theta) = \mathbb{E}_{t, x_0, x_1} [\| v_{\theta}(x_t, t) - u_t \|^2]$$

训练范式差别 → 训练成本相差 100-400×。

背景：现有 PDE 神经求解三大流派

(2) Neural ODE (CNF)

使用NeuralODE进行完整轨迹建模（多点监督）：学习 $u_t \rightarrow u_{t+1}$

训练 loss：观测匹配（以 PDE 轨迹为例， pairwise teacher forcing）

$$\mathcal{L}_{NODE}(\theta) = \frac{1}{N} \sum_{i=0}^{N-1} \left\| \underbrace{u(t_i) + \int_{t_i}^{t_{i+1}} f_{\theta}(\tau, u(\tau)) d\tau}_{= ODEsolve(f_{\theta}, u(t_i) [t_i, t_{i+1}])} - u(t_{i+1}) \right\|^2$$

外层是普通求和；真正的计算成本全部藏在"每个内层 ODEsolve"里。

实际任意连续时间位置推理：

查询	标准 NODE (端点监督)	多点监督 NODE
查询 $u(t+0.3)$	可以输出,但 真实上不可信	可以输出,且 真实轨迹上较可信
保证	只保证 $u(t+1)$ 准确	保证所有被监督的点准确

背景：现有 PDE 神经求解三大流派

(3) 生成模型 Diffusion

- **PDE-Refiner**：扩散校正神经算子误差
- **DiffusionPDE**：物理约束扩散求解
- **CoCoGen**：物理一致 score-based
- **PBFM**：FM + 物理 conflict-free 梯度

优点：

- 表达力强（生成模型）
- 可处理 partial observation / inverse
- 不确定性建模天然支持

问题：

- 推理慢（多步去噪/采样）
- 通常需要显式 PDE 形式（如 DPS）

补充：Gencp 耦合物理场建模

方法：一次性生成所有轨迹

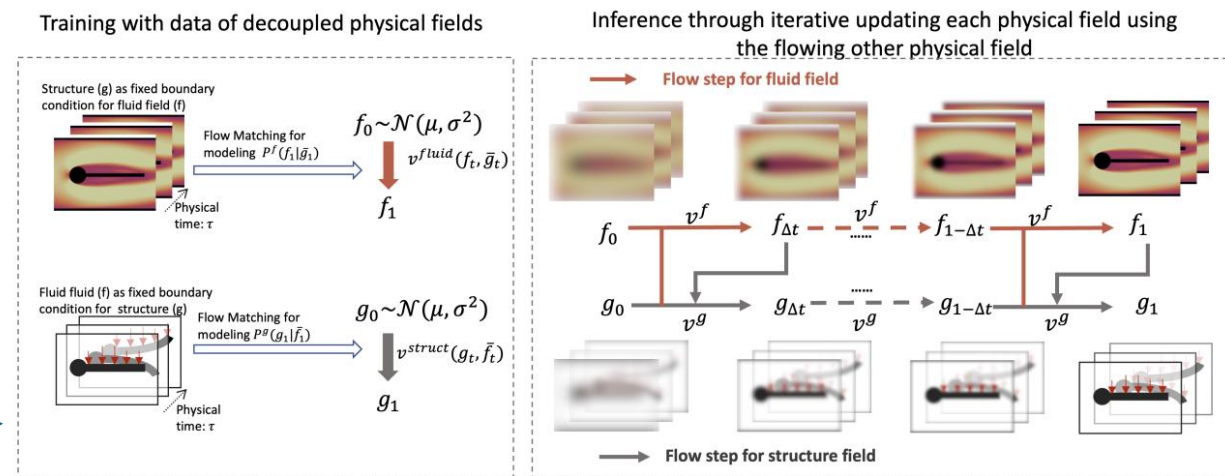
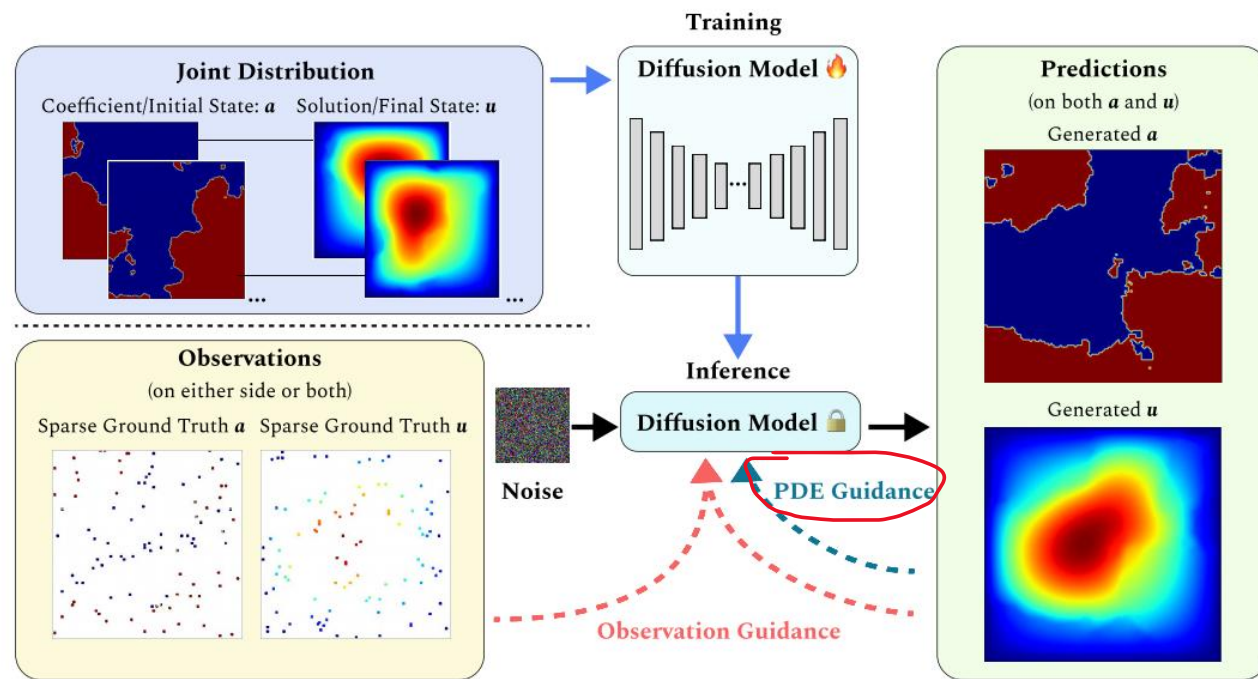


Figure 1: The schematic of GenCP. With a pretrained model with decoupled physics, GenCP could manage to achieve coupled physics during flow steps. Note that here we only use Lie-Trotter Splitting as a demonstration.

CFO: Learning Continuous-Time PDE Dynamics via Flow-Matched Neural Operators (ICLR 2026)

论文主线叙事:

痛点 1: 时间维度处理不优雅

- AR: 离散步骤, 误差累积
- Operator learning: 固定时间网格
- 都不能任意时间查询

痛点 2: 训练成本与精度的 trade-off

- 想要连续时间 → 用 Neural ODE → 训练慢 100-400×
- 想要训练快 → 用 AR → 长时程发散

痛点 3: 物理一致性差

- AR 不知道"PDE 动力学", 只知道"下一步"
- 错误率随 horizon 指数累积

CFO:

(1) ODE

(2) Flow Matching

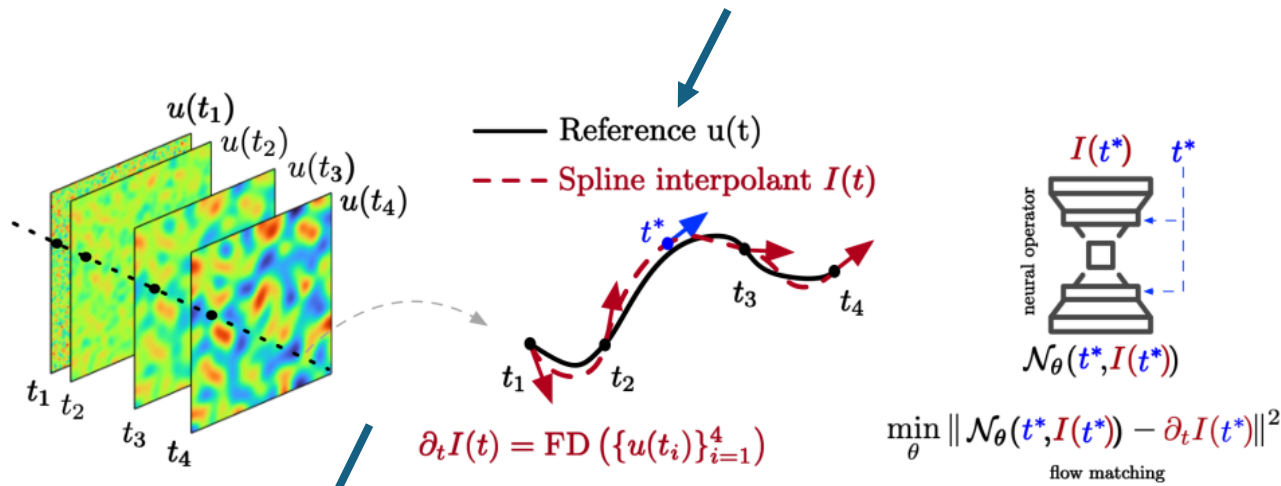
(3) 无显式PDE方差, 纯数据驱动

CFO: 使用Flow Matching 实现数据到数据的连续动态演变

FM 框架的灵活性来自'路径设计'

CFO： **PDE 的真实物理演化轨迹本身就是一条天然路径，直接用！**

维度	经典 FM	CFO
起点	噪声（人造）	数据（PDE 初值，真实）
终点	数据	数据（PDE 终态）
路径	人造（线性插值等）	真实 PDE 轨迹（spline 拟合）
时间 t	归一化 [0,1]	物理时间（秒、小时等）



框架核心方法： (a) Training 网络结构： FNO、Unet都可以

怎么得到数据演化路径 (FM中是概率路径)

CFO: 使用Flow Matching 实现数据到数据的连续动态演变

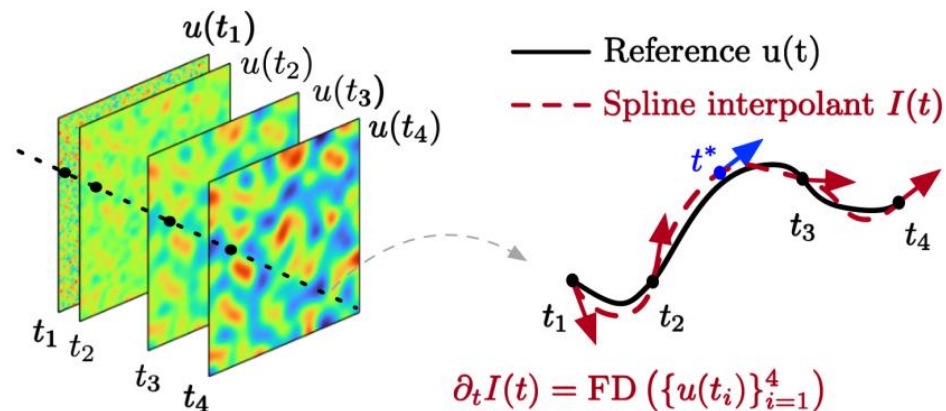
训练数据只是离散观测点 $\{u(t_0), u(t_1) \dots u(t_N)\}$, 需要先在观测点之间插值出连续可微的代理轨迹:
对应Flow Matching的 **Probability Path**:

$$I(t; \mathbf{u}) = s(t; \mathbf{u}) + \gamma(t) z, \quad t \in [0, 1],$$

要求:

1. 插值条件: $s(t_i; \mathbf{u}) = u(t_i), \quad i = 0, \dots, N.$

2. 轨迹全局光滑



(1) 有限差分近似 — knot 处导数估计

一阶导数 (速度) :
$$d(t_i) = -\frac{h_i}{h_{i-1}(h_{i-1} + h_i)}u(t_{i-1}) + \frac{h_i - h_{i-1}}{h_{i-1}h_i}u(t_i) + \frac{h_{i-1}}{h_i(h_{i-1} + h_i)}u(t_{i+1}),$$

二阶导数 (加速度) :
$$a(t_i) = \frac{2}{h_{i-1} + h_i} \left(\frac{u(t_{i+1}) - u(t_i)}{h_i} - \frac{u(t_i) - u(t_{i-1})}{h_{i-1}} \right).$$

两个三点的加权和计算速度

两个相邻段的"斜率差"

CFO：使用Flow Matching 实现数据到数据的连续动态演变

(2) Segment Interpolation — 段内 Hermite 插值

想要全局 C^s 光滑 (s 阶导也连续)，就需要在每个 knot 强制 s 阶以内的导数连续

想要 C^s 光滑	每端匹配阶数	多项式最小阶	名称
C^0 (位置连续)	仅 value	1	Linear
C^1 (速度连续)	value + 1 阶导	3	Cubic Hermite
C^2 (加速度连续)	value + 1 阶导 + 2 阶导	5	Quintic Hermite ★

线性插值: $s(t; \mathbf{u}) = \frac{t_{i+1} - t}{h_i} u(t_i) + \frac{t - t_i}{h_i} u(t_{i+1}).$

闭式的线性系统

五次样条插值： 6 个约束 → 6 个系数 → 唯一确定 5 次多项式：

$s(t) |_{[t_i, t_{i+1}]} = c_0 + c_1(t - t_i) + c_2(t - t_i)^2 + c_3(t - t_i)^3 + c_4(t - t_i)^4 + c_5(t - t_i)^5$

$$\begin{cases} c_0 = p_0 \\ c_1 = v_0 \\ 2c_2 = a_0 \\ c_0 + c_1h + c_2h^2 + c_3h^3 + c_4h^4 + c_5h^5 = p_1 \\ c_1 + 2c_2h + 3c_3h^2 + 4c_4h^3 + 5c_5h^4 = v_1 \\ 2c_2 + 6c_3h + 12c_4h^2 + 20c_5h^3 = a_1 \end{cases}$$

位置 + 速度 + 加速度 在所有 knot 都连续。

——物理直觉： 匹配物理定律（牛顿、波动）

CFO: 使用Flow Matching 实现数据到数据的连续动态演变

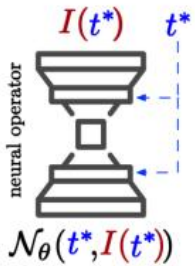
(3) FM的概率路径

$$I(t; \mathbf{u}) = s(t; \mathbf{u}) + \gamma(t) z, \quad t \in [0, 1],$$

插值轨迹

噪声幅度

$$\gamma_0 \frac{(t-t_k)^m (t_{k+1}-t)^m}{(t_{k+1}-t_k)^{2m}}, \quad t_k \leq t < t_{k+1}$$



$$\min_{\theta} \|\mathcal{N}_{\theta}(t^*, I(t^*)) - \partial_t I(t^*)\|^2$$

flow matching

$\sigma(\tau=0)=\sigma(\tau=1)=0 \longrightarrow$ 路径精确通过数据点（不退化）

中间 $\sigma_t > 0$ ("鼓包"形) $\longrightarrow$ 推理时有点偏差，仍在管道内，网络能正确引导

(4) 训练

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{u}, t, Z} \left[\|\mathcal{N}_{\theta}(t, I(t; \mathbf{u})) - \partial_t I(t; \mathbf{u})\|^2 \right],$$

CFO: 正向推理与反向推理

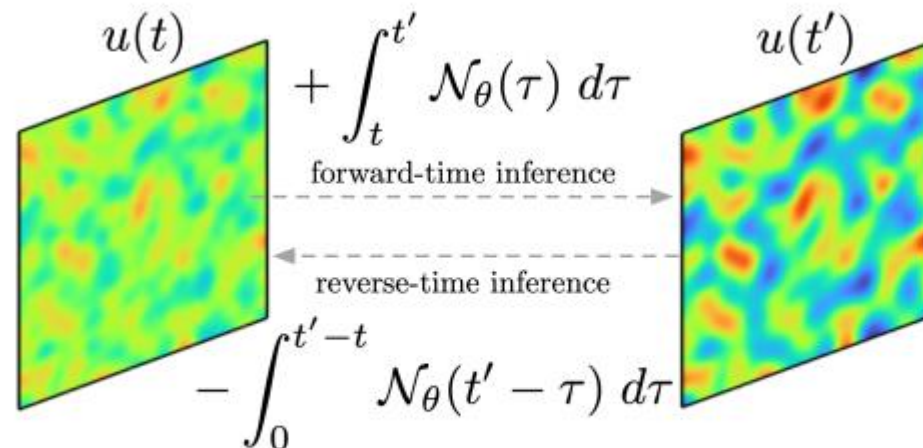
正向轨迹推理: $u(t) = u(0) + \int_0^t v_\theta(u(s)) ds$

每个 Δt 分两段做FM, 每段用RK4数值积分 (总共8次FM)

每段用 4 个不同位置的FM计算出的斜率加权平均, 得到更精确的步进方向:

$$\begin{aligned}k_1 &= f(t_n, u_n) \\k_2 &= f(t_n + \frac{h}{2}, u_n + \frac{h}{2}k_1) \\k_3 &= f(t_n + \frac{h}{2}, u_n + \frac{h}{2}k_2) \\k_4 &= f(t_n + h, u_n + hk_3) \\u_{n+1} &= u_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)\end{aligned}$$

反向轨迹推理 $u_\theta(s) = u(t_\star) + \int_{t_\star}^s \mathcal{N}_\theta(\tau, u_\theta(\tau)) d\tau = u(t_\star) - \int_0^{t_\star-s} \mathcal{N}_\theta(t_\star - \tau, u_\theta(t_\star - \tau)) d\tau,$



(b) Inference

CFO: 实验

Baseline（单步预测 + 自回归 rollout）: (基于MLP、 U-Net 、 FNO 的Teacher Forcing架构)取最好的结果作为 AR Baseline

L2 损失

训练时候完整PDE轨迹的3 种采样密度： 100% / 50% / 25%

Dataset	Sampling	Autoregressive	LinearCFO	QuinticCFO
Lorenz	100%	$(9.04 \pm 2.80) \times 10^{-2}$	$(6.42 \pm 0.69) \times 10^{-2}$	$(4.53 \pm 0.68) \times 10^{-2}$
	50%	—	$(7.85 \pm 0.25) \times 10^{-2}$	$(4.75 \pm 0.78) \times 10^{-2}$
	25%	—	$(9.39 \pm 0.77) \times 10^{-2}$	$(6.82 \pm 0.48) \times 10^{-2}$
Burgers'	100%	$(3.34 \pm 0.14) \times 10^{-2}$	$(5.75 \pm 0.72) \times 10^{-3}$	$(5.89 \pm 0.79) \times 10^{-3}$
	50%	—	$(8.91 \pm 1.30) \times 10^{-3}$	$(6.97 \pm 0.73) \times 10^{-3}$
	25%	—	$(1.04 \pm 0.15) \times 10^{-2}$	$(7.09 \pm 1.10) \times 10^{-3}$
DR	100%	$(4.23 \pm 0.59) \times 10^{-1}$	$(4.35 \pm 0.67) \times 10^{-2}$	$(4.37 \pm 0.47) \times 10^{-2}$
	50%	—	$(6.88 \pm 0.49) \times 10^{-2}$	$(6.19 \pm 0.35) \times 10^{-2}$
	25%	—	$(7.25 \pm 0.48) \times 10^{-2}$	$(5.32 \pm 0.65) \times 10^{-2}$
SWE	100%	$(9.04 \pm 0.36) \times 10^{-2}$	$(5.93 \pm 0.95) \times 10^{-3}$	$(4.56 \pm 0.38) \times 10^{-3}$
	50%	—	$(7.64 \pm 0.52) \times 10^{-3}$	$(6.57 \pm 0.66) \times 10^{-3}$
	25%	—	$(1.69 \pm 0.38) \times 10^{-2}$	$(1.55 \pm 0.22) \times 10^{-2}$

CFO: 线性插值与全局样条插值的对比（在Lorenz 系统的已知轨迹数据上测试）

(1) 插值时的速度轨迹误差（轨迹先验本身误差）：

线性插值在端点误差很大（速度突变）

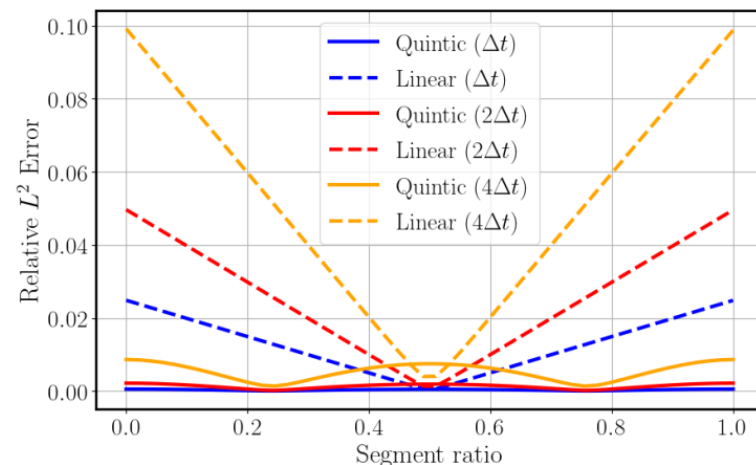


Figure 2: Spline fitting error on the Lorenz system. Average velocity-field approximation error for quintic and linear splines at different time-step sizes (Δt , $2\Delta t$, $4\Delta t$), with $\Delta t = 0.005\text{s}$.

(2) 训练损失

采样点越稀疏：

quintic 显著更稳定、收敛更快

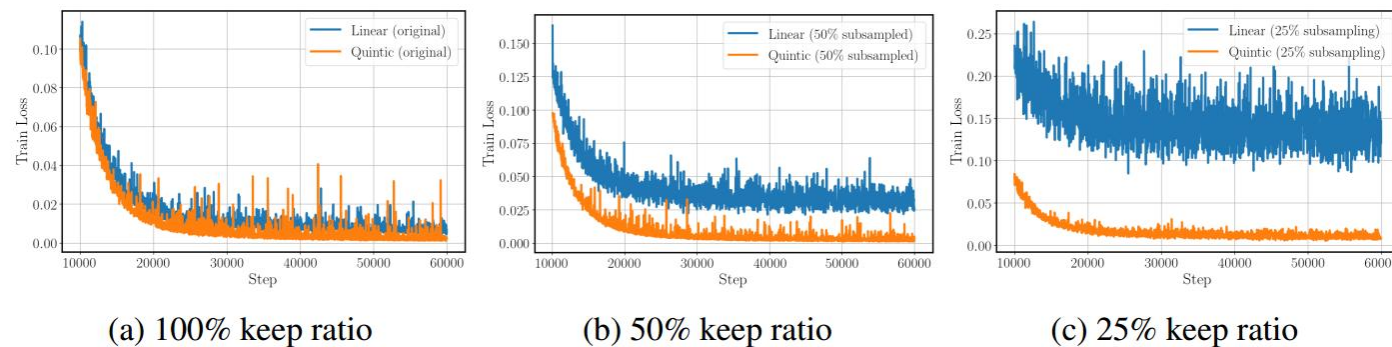


Figure 3: Training comparison between linear and quintic CFO on the Lorenz system. Training loss (10k–60k iterations) for linear and quintic CFO under different keep ratios: (a) 100%, (b) 50%, (c) 25%.

CFO: 长时程稳定性可视化

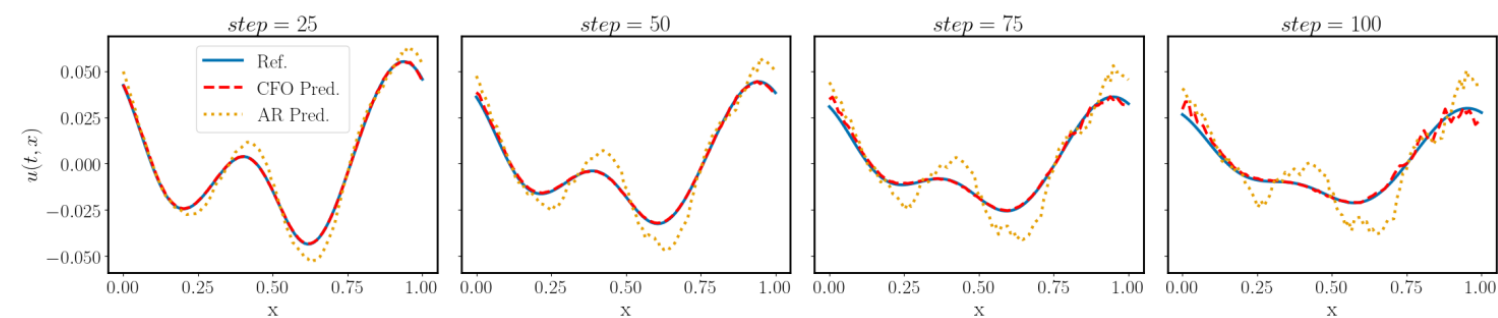
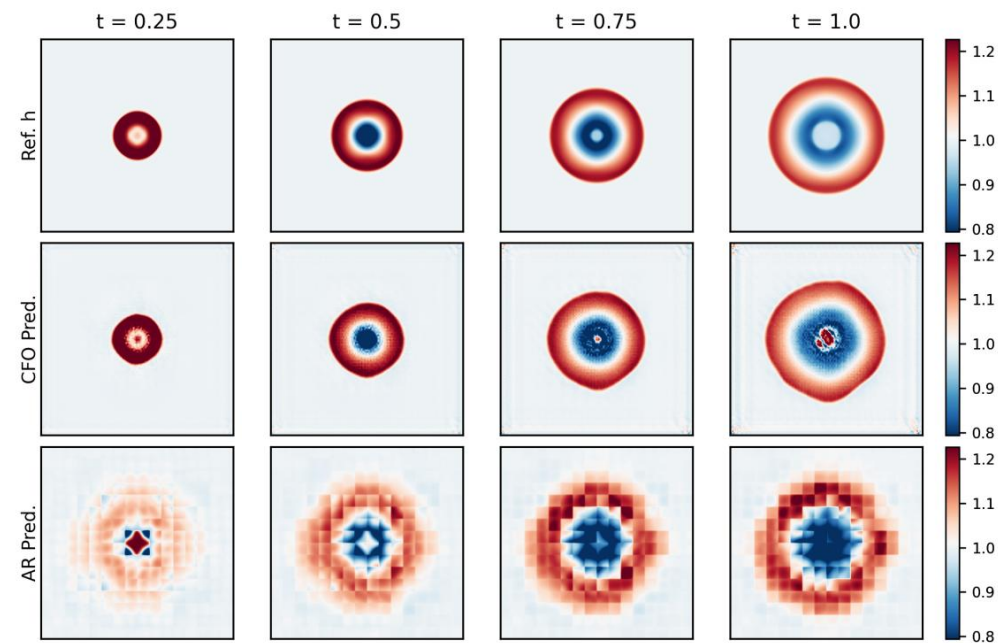


Figure 4: Worst-case prediction for Burgers’ equation. Quintic CFO (trained on 25% irregular data) accurately predicts the full trajectory for the test sample with the highest error. Shown from left to right: solutions at the 25, 50, 75, and 100th time steps.



CFO：推理时间

(1) NFE 推理次数

50%的NFE就能达到更好的精度

Table 2: Influence of NFE on quinticCFO (RK4 solver) accuracy (Relative L^2 Error) at the final time step, compared to the autoregressive baseline.

NFE (% AR)	Lorenz	Burgers'	DR	SWE
AR (100%)	0.1481	0.0647	0.3850	0.1048
50%	0.0919	0.0090	0.0876	0.0698
100%	0.0655	0.0089	0.0718	0.0154
200%	0.0674	0.0090	0.0608	0.0060
400%	0.0645	0.0089	0.0593	0.0061

(2) 训练时间

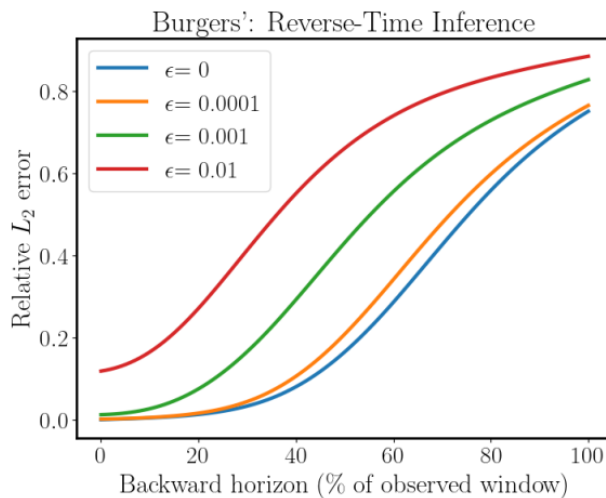
对比NeuralODE和Diffusion

Table 3: Comparison of CFO with other baselines: Neural ODE (Chen et al., 2018) and PDE-Refiner (Lippe et al., 2023).

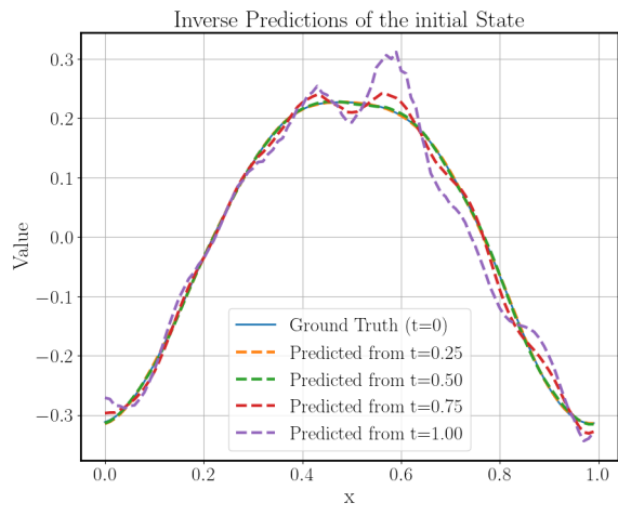
Equation	Method	Rel. L^2 Error	Training Time (s/batch)
Lorenz	Neural ODE	0.101	0.133
	CFO	0.0453	0.00350
Burgers	Neural ODE	0.0275	3.38
	CFO	0.00589	0.00920
DR	PDE-Refiner	0.125	1.38
	CFO	0.044	0.40
SWE	PDE-Refiner	0.093	1.40
	CFO	0.005	0.40

CFO: 反向轨迹推导

短 horizon (20%) : 误差 ~ 0.05 , 可用
全 horizon (100%) : 误差 ~ 0.80 , 完全失败



(a) Relative L_2 error of reconstructed $\hat{u}(t)$ for $t \in [0, 1)$ under different noise levels added to the terminal state $u(1)$.



(b) Trajectories reconstructed backward from $t \in \{1, 0.75, 0.5, 0.25\}$ to $t = 0$.

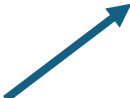
Figure 11: Reverse-time inference for dissipative Burgers' equation.

CFO: 分析

- (1) 耗散 PDE 反向时间天然 ill-posed (物理本质)
- (2) 反问题的本质是 Bayesian inverse, 需要后验分布, CFO 给的是点估计。
- (3) 训练只见过正向时间的状态 (NeuralODE理论上反问题求解更好)
- (4) 噪声/误差被反向积分指数放大

能不能/值不值得改进?

维度	FNO / DeepONet	Neural ODE	Diffusion / Generative	CFO ★
时间分辨率	固定离散	任意连续	任意连续	任意连续 ✓
训练速度	快	✗ 慢 (adjoint)	中	快 ✓
推理速度	极快 (1 forward)	中 (ODE 步)	✗ 慢 (多步采样)	中 (~8× AR NFE)
长时程稳定	✗ 差 (AR 累积)	好	△分情况	好 ✓
训练稳定	好	✗ 差 (adjoint 震荡)	△不好说	好 ✓
是否需要准确 PDE 表达式 ★	✗ 不需要	✗ 不需要	△ 看方法 (DPS 需要, PDE-Refiner 不需要)	✗ 不需要 ✓
是否需要全轨迹数据 ★	✓ 需要全场数据	✓ 需要密集快照	✓ 需要全场数据	天然支持稀疏、不规则数据 ✓
物理一致性 (可解释性)	弱	中	强 (带 PDE guidance)	中-强 ✓
反问题支持	弱	中 (adjoint)	强 (DPS)	弱 (仅短 horizon)


和Flow Matching训练模式有关
不过CFO主线聚焦于前向过程

恢复完整正向轨迹动态出发的物理场求解问题:

GenCP 一次性扩散生成完整轨迹

- 高维时许同时生成-参数量极大
- 缺乏解释性
- 且Diffusion需要前几帧状态约束扩散方向

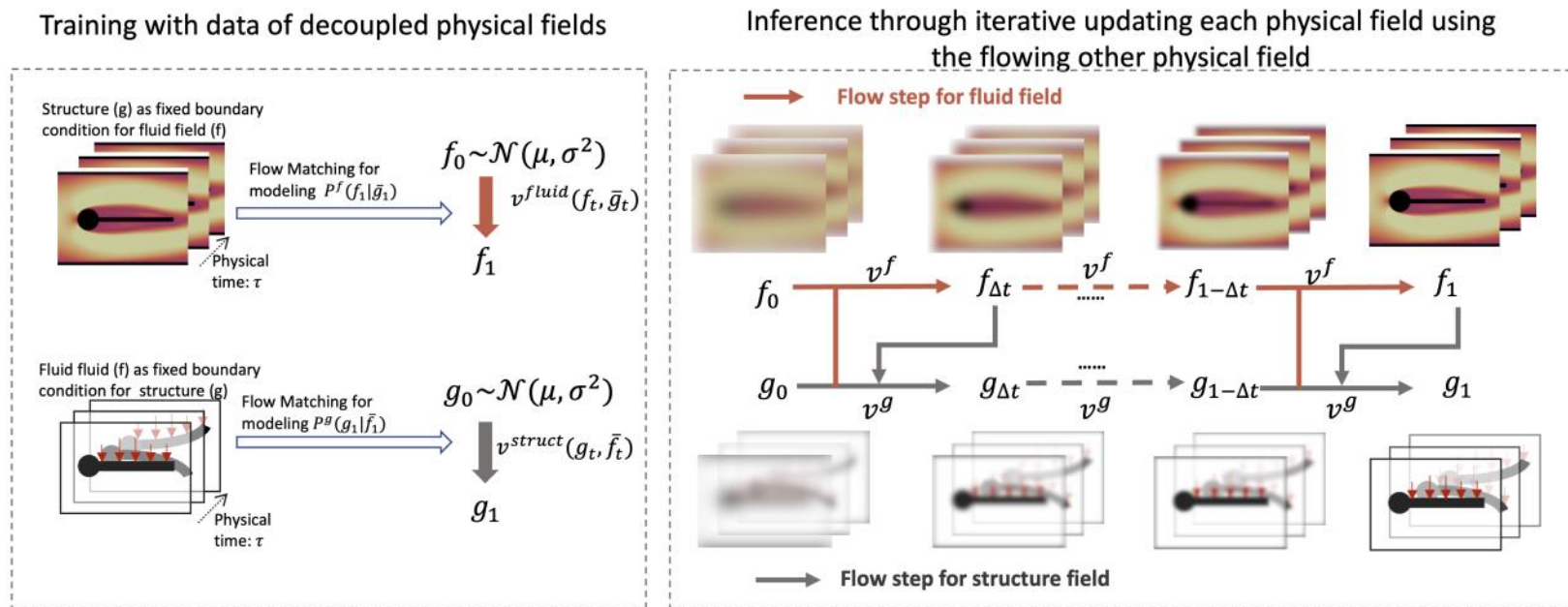
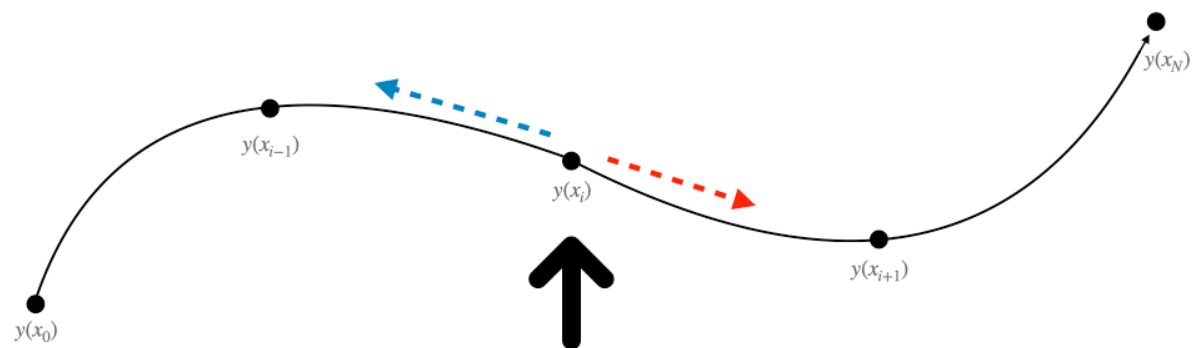


Figure 1: The schematic of *GenCP*. With a pretrained model with decoupled physics, *GenCP* could manage to achieve coupled physics during flow steps. Note that here we only use Lie-Trotter Splitting as a demonstration.

恢复完整轨迹动态出发的物理场求解问题：

数据同化任务：多点观测下的adjoint轨迹优化



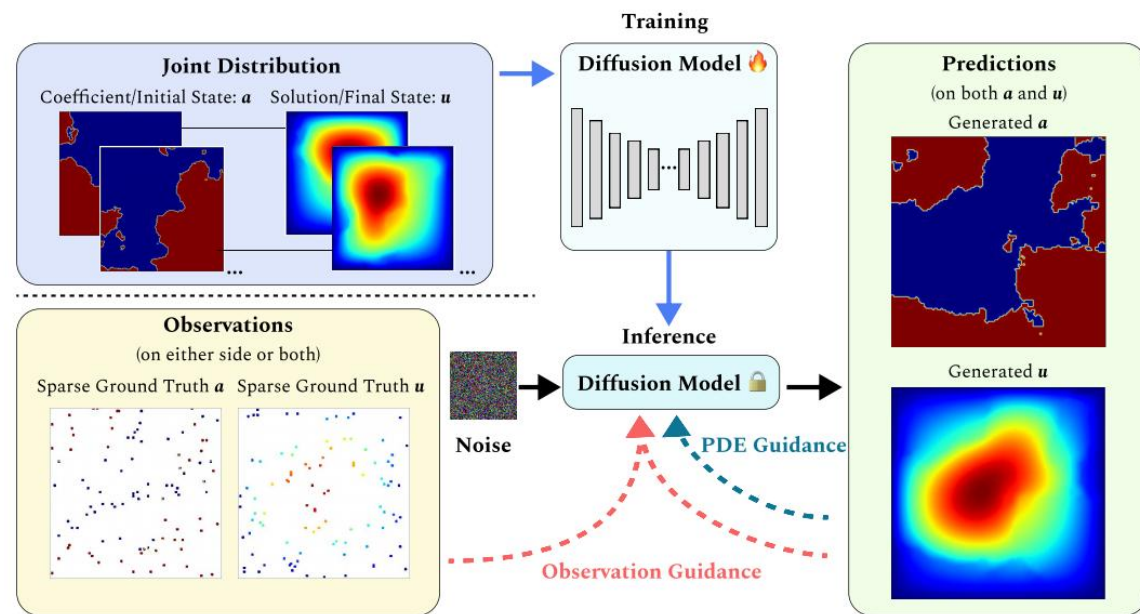
反向完整轨迹动态推导任务？：

DiffusionPDE：

学 $p(u_0, u_T)$ 的联合分布，PDE Guidance进行双向约束——无完整轨迹学习，把"中间"留给传统数值方法（有了initial state可以用PDE恢复整个轨迹）

在没有PDE情况下？怎么做类似DPS的任意位置反轨迹？

似乎做不了（绕不开ill pose问题）



Bidirectional 训练双向显式轨迹的FM

- 不过无法突破本身反向ill-posed的物理本质，还是需要结合后验概率